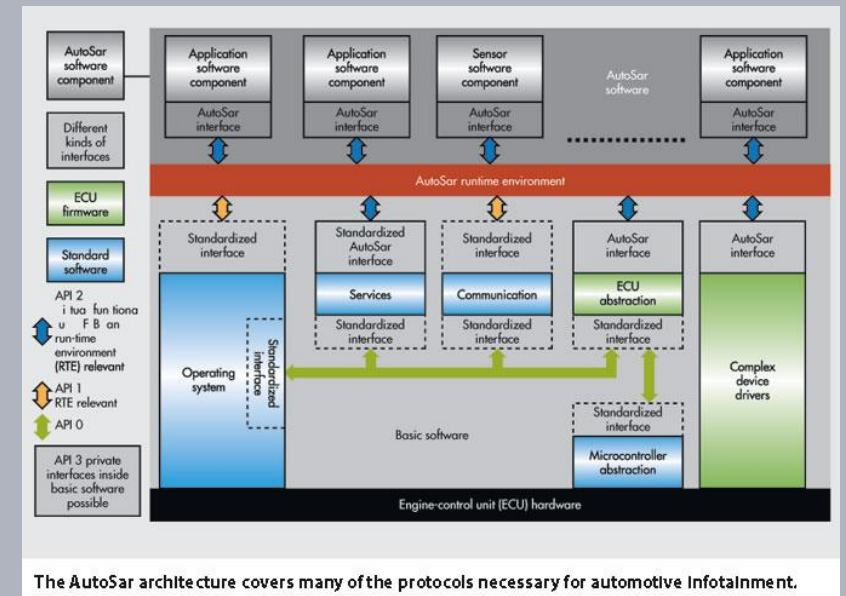


Advanced Microcontroller Architectures and Embedded OS

Task Communication

Prof. Dr.-Ing. Peter Fromm



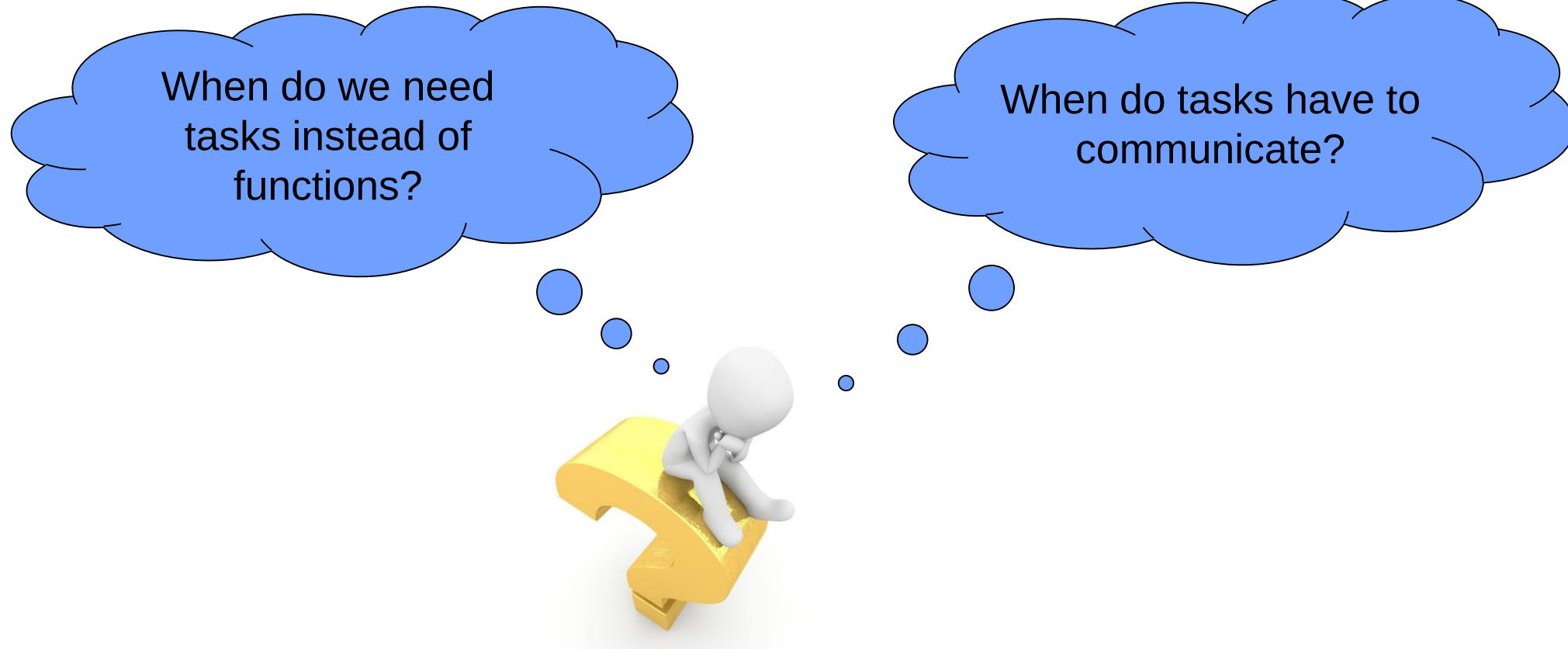
The AutoSar architecture covers many of the protocols necessary for automotive Infotainment.

Content

- Look at decoupled versus coupled systems
- Learn about synchronization and data transfer between tasks
- Understand the OSEK Event mechanism
- Have a look at messages

Design criteria for tasks

A **task** is a **function** called by the **scheduler**.



Design considerations

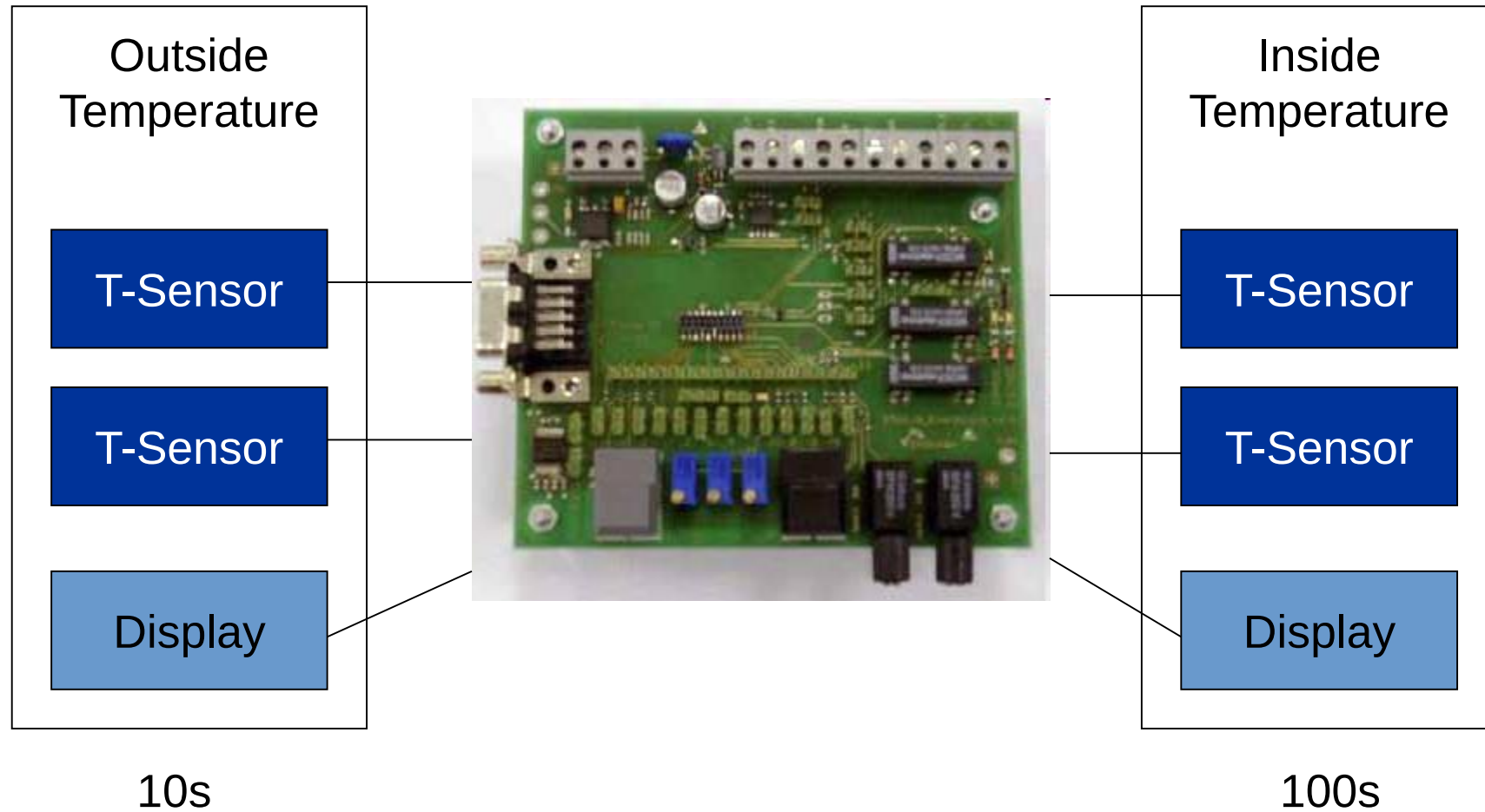
Motivation for tasks

- Processes run independently and / or at the same time
- Processes only need to be active under certain conditions
- Processes can be pre-empted by higher priority tasks
- Processes need to protect their resources (memory, peripherals)

Motivation for task communication

- Access to shared resources (e.g. shared peripherals or memory)
- Application specific dependencies (e.g. on button press wake up)
- Temporal dependencies (e.g. cyclic activations)

Example: Independent Processes

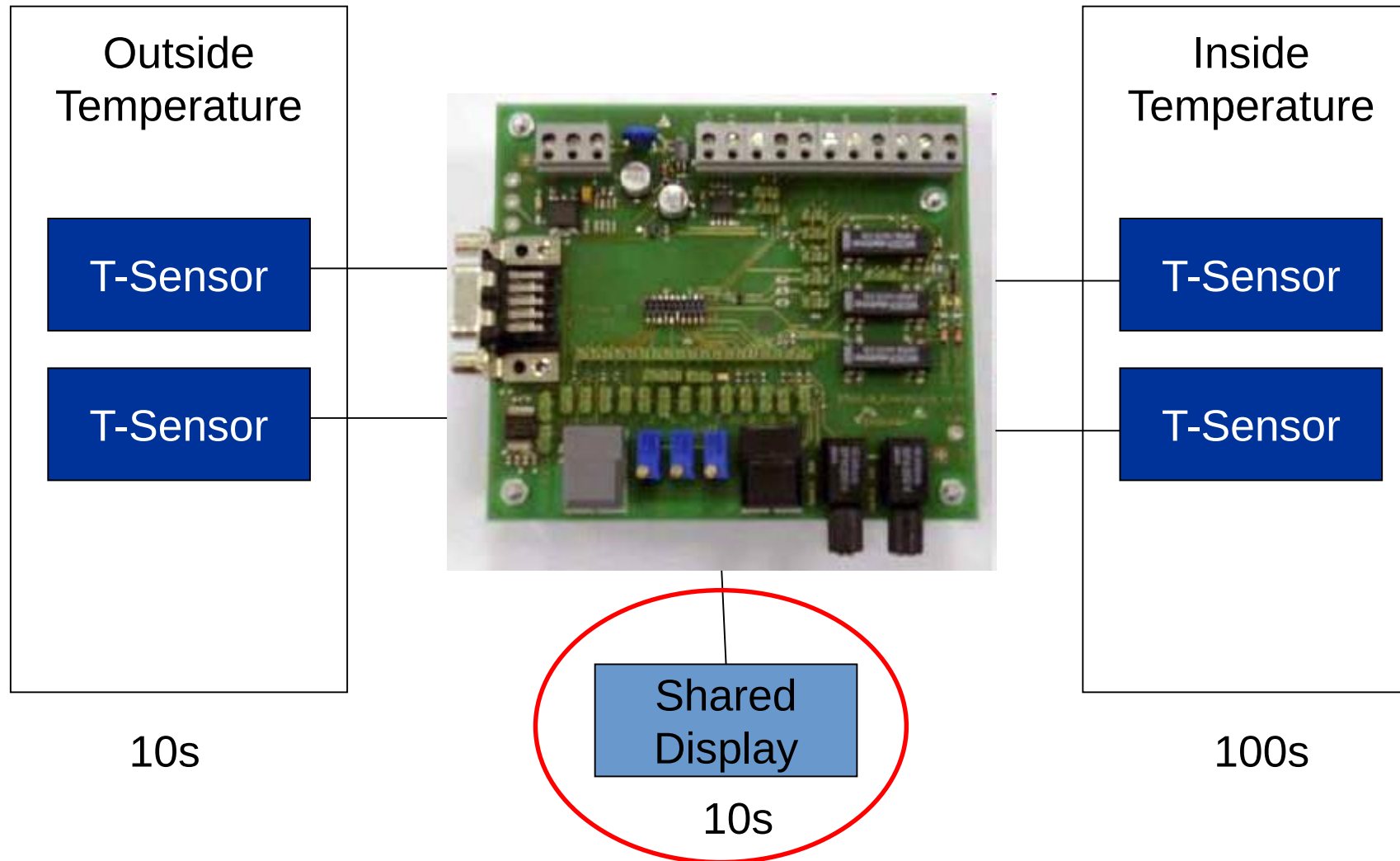


Example: Independent Processes

Considering

- A decoupled architecture
- The MCU controls a set of independent devices, consisting of sensors and actors
- Every device is controlled by an own task
- ➔ The OS only has to make sure that the tasks are called within the agreed cycle time
- ➔ Failure of one device does not harm the others
- ➔ Because the devices are independent, they can be interrupted and resumed without any constraints

Example: Coupled Processes – Shared Display



Example: Coupled Processes – Shared Display

In case resources are shared (which usually is the case)

→ a task can only access a resource as long as no other task uses it

Coupled systems can be identified by

- Shared resources
- Timing constraints (before task B can be executed, task A has to deliver a result or has to reach a certain state)

Note: Multithreading systems always share the global memory and the heap. Global variables as a consequence always are shared resources!
→ Data Consistency

Sharing resources – synchronous calls

Synchronous operation:

```
outsideTemp.sendProtocol(0x45, "12deg");  
insideTemp.sendProtocol(0x45, "19deg");
```



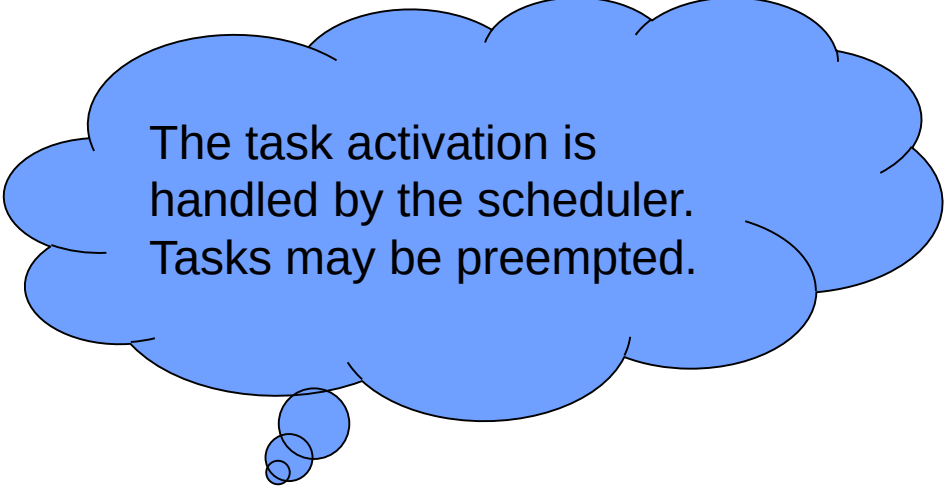
Synchronous operation:

- Release time of function is identical to the function call.
- Release of the next function is automatically after end time of previous function

Sharing resources in tasks

Function call:

```
TASK (outsideTemp)
{
    sendProtocol (0x45, "12deg");
}
TASK (insideTemp)
{
    sendProtocol (0x45, "19deg");
}
```

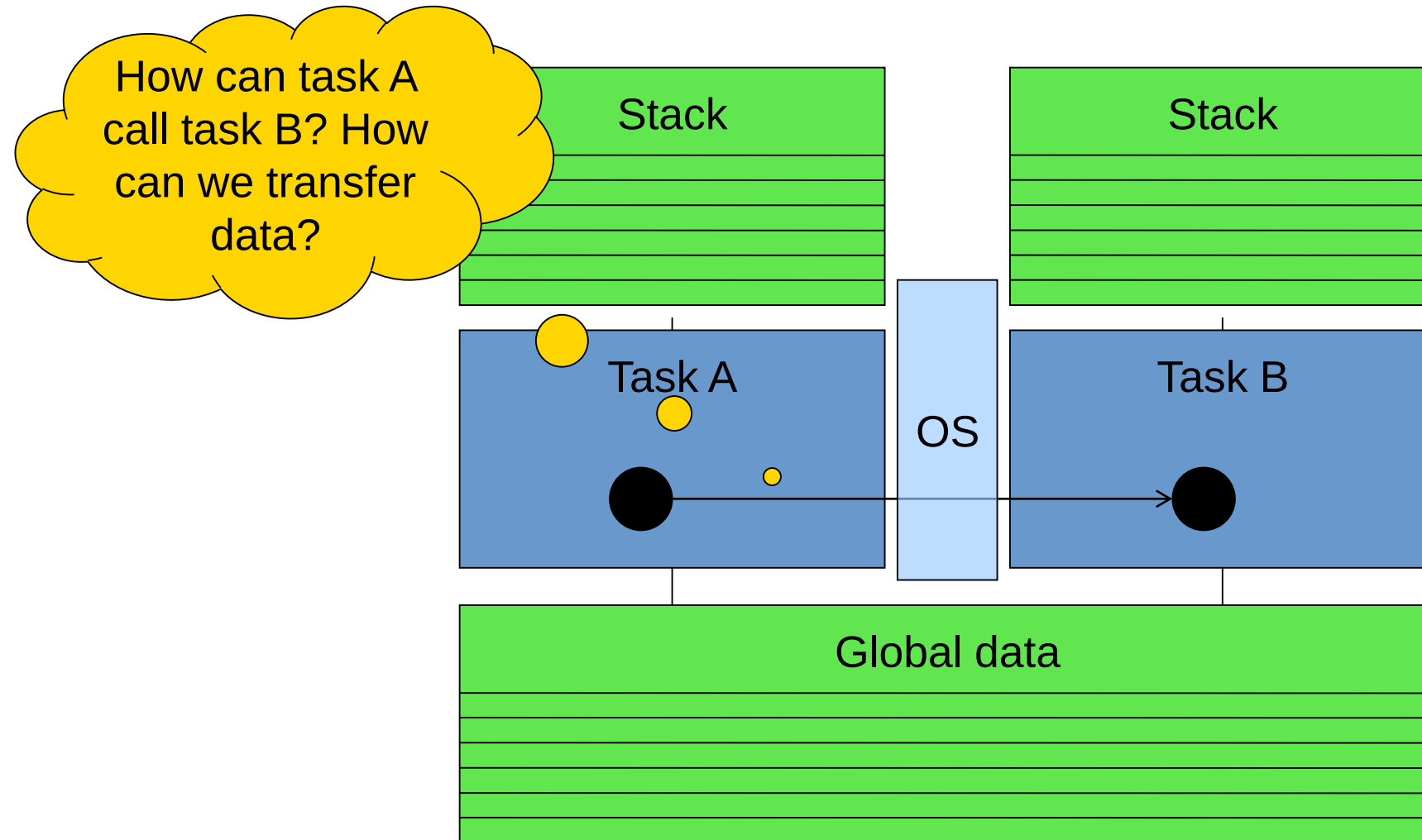


The task activation is
handled by the scheduler.
Tasks may be preempted.

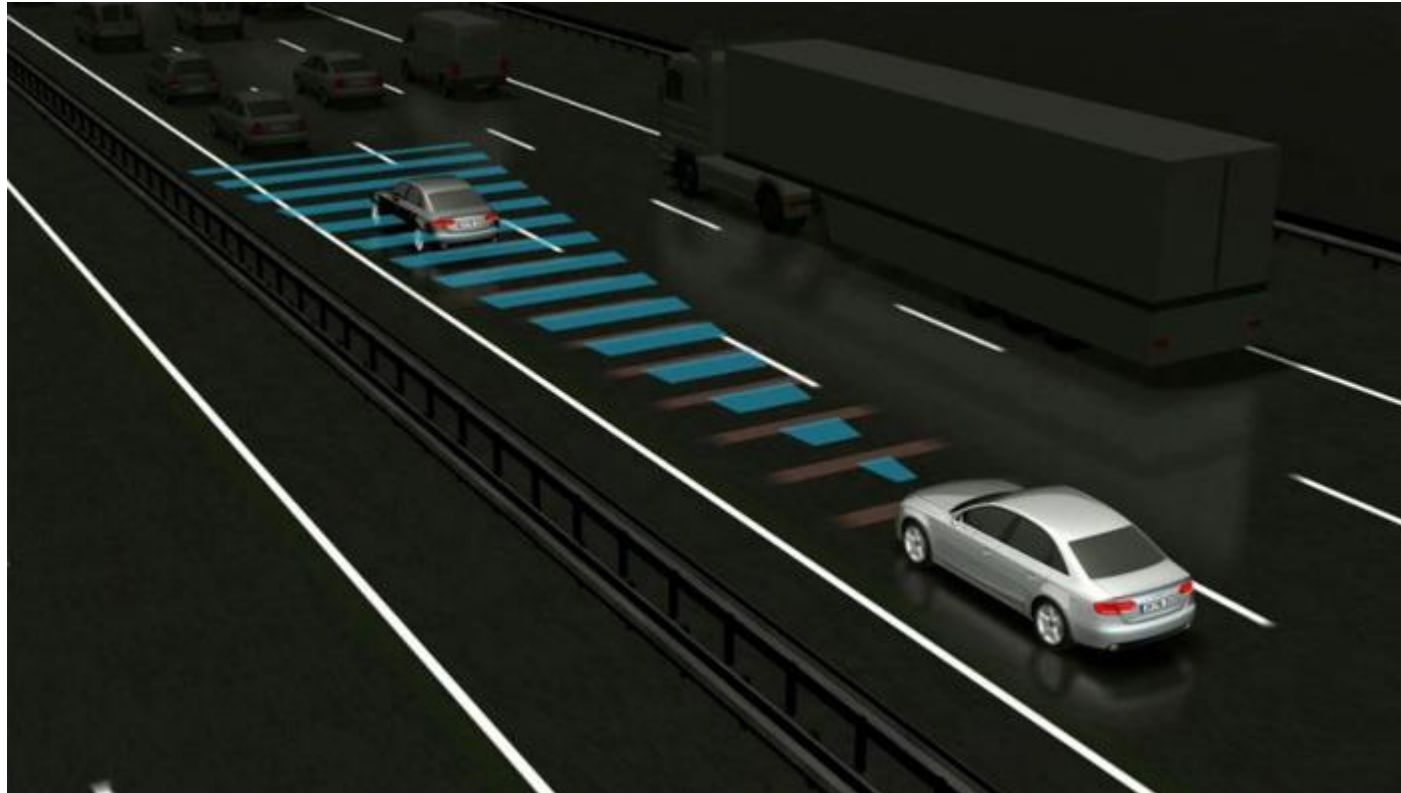
This results in a couple of requirements:

- A task must know whether a resource is in use or not
- A mechanism must be provided which allows to send a task “on holiday” (Suspend)
- And a mechanism is required to get the task back “to work” (resume)
- And we need a functionality to transfer data....

„Calling“ a task



Example: Adaptive Cruise Control

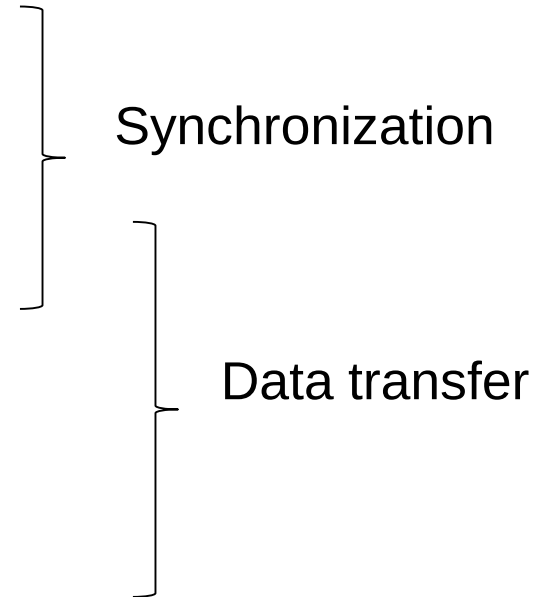


Source: Audi

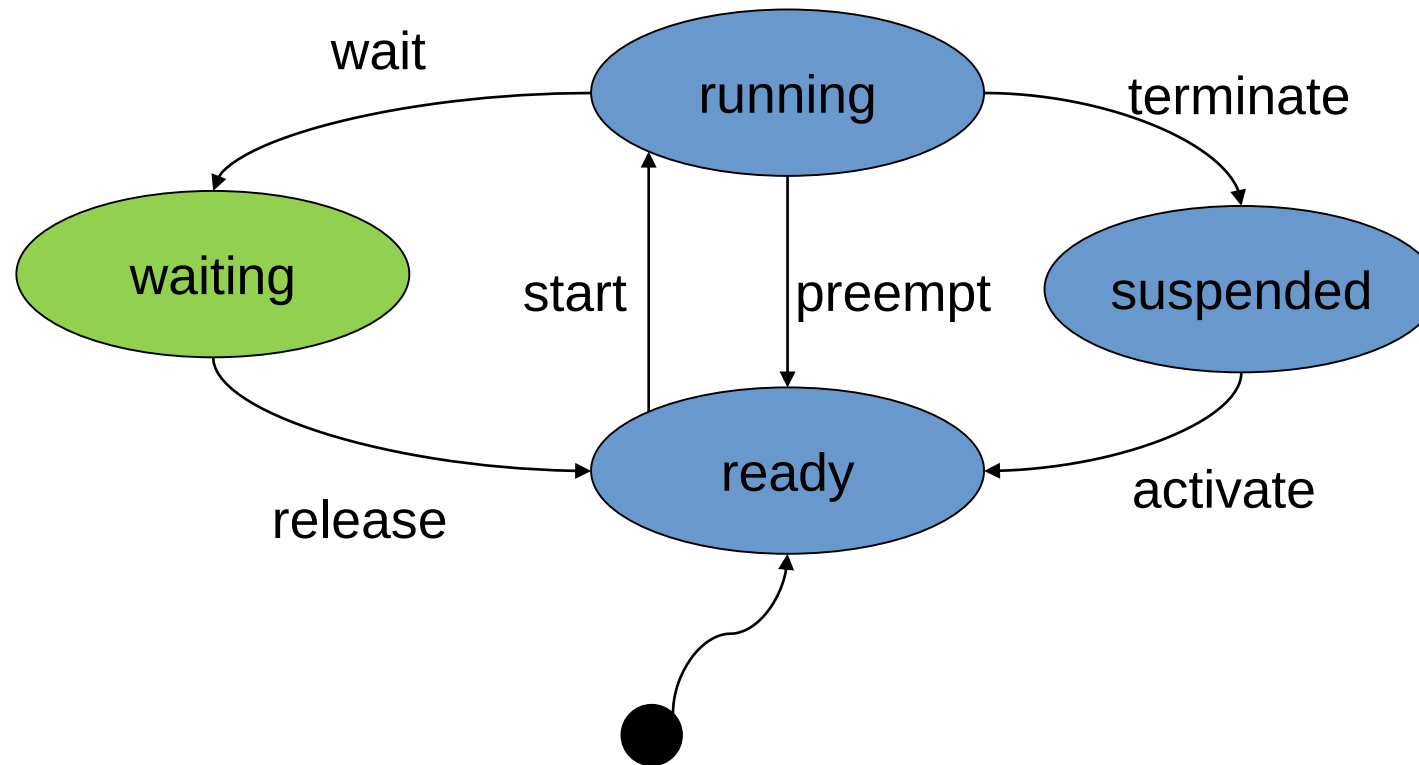
Synchronisation Mechanisms

Some techniques

- Semaphores, **Mutexes**
- **Events** or Signals
- **Alarms**
- Remote Procedure Calls
- Messages
- **Shared (protected) memory areas**
- Shared files
- Pipes
- Streams



Task states - OSEK



OSEK Events

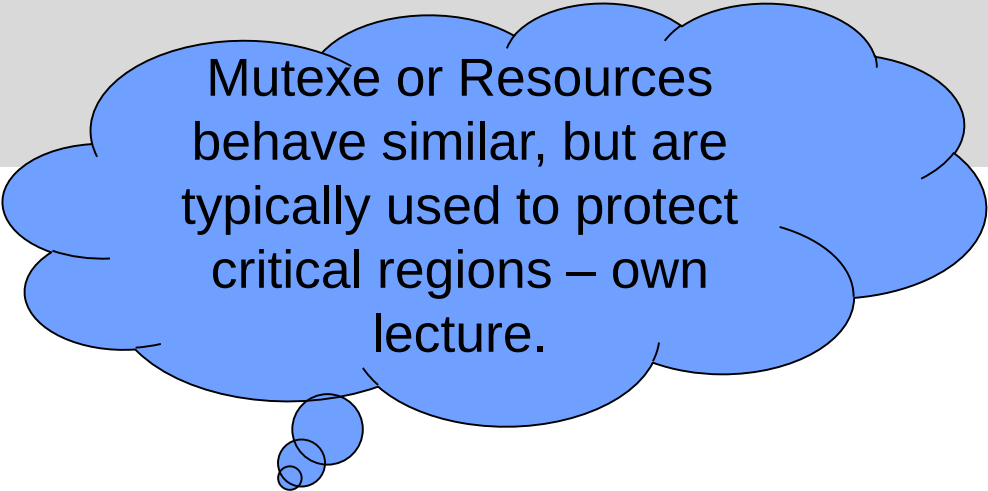
```
TASK (Delivery_TASK)
{
    EventMaskType myEvents;

    while(1) {

        // Wait for an event
        WaitEvent(ev_a | ev_b | ev_c);
        GetEvent(Delivery_TASK, &myEvents);
        ClearEvent (myEvents);

        if (myEvent & ev_a) // Process the event ev_a
        if (myEvent & ev_b) // Process the event ev_b
        if (myEvent & ev_c) // Process the event ev_c

    }
}
```

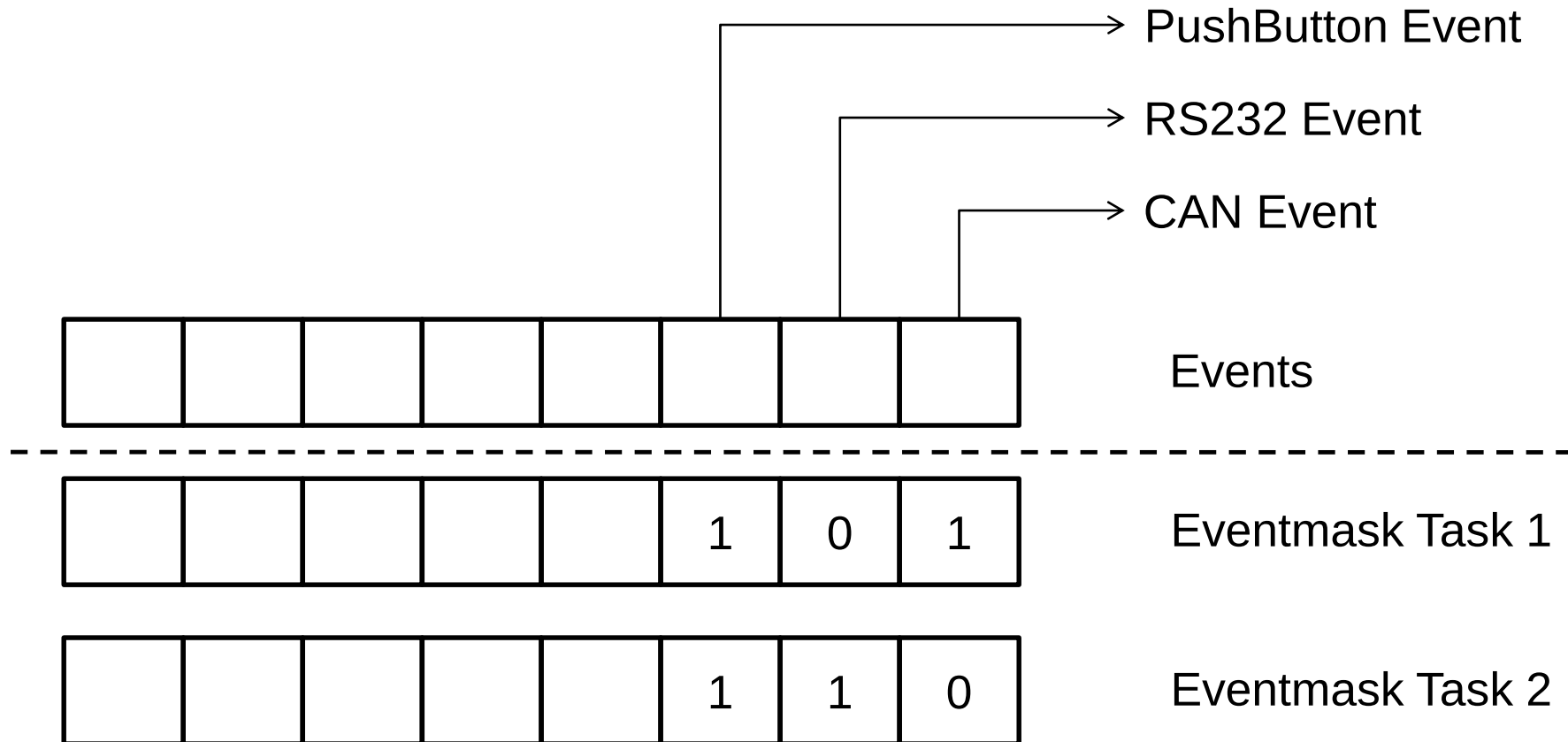


Mutexes or Resources
behave similar, but are
typically used to protect
critical regions – own
lecture.

OSEK Events

- Tasks can wait for events to occur (WaitEvent)
- Tasks can use events to signal a state change
- A waiting task can check the occurrence of an event, but it does not know who sent it
- Only extended tasks may wait for events (because they have a pending state)
- Every extended task can wait for one or more events
- Events can be generated by
 - Tasks (basic and extended)
 - Alarms
 - Category 2 ISR
 - Incoming or outgoing messages (OSEK/COM)

OSEK Events



*OSEK support 8, 16 or 32 events

OSEK Events

OSEK event API

- EventMaskType → Type to store eventmask
- DeclareEvent(inEventID) → Declaration of an event
- SetEvent(inTaskID, inEventMask) → Set event inEventMask for Task inTaskId
- GetEvent(inTaskID, outEventMask) → Get event outEventMask for Task inTaskID
- ClearEvent(inEventMask) → Delete the events specified by inEventMask
- WaitEvent(inEventMask) → Wait for the event specified by inEventMask

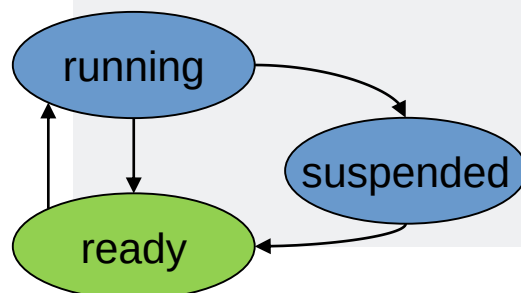
OSEK Event Sequence (1)

TASK(Sender)

//Execute some code

SetEvent(Receiver, ev1);

//Execute some code



TASK(Receiver)

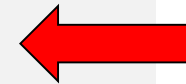
//Execute some code

WaitEvent(ev1|ev2|ev3);

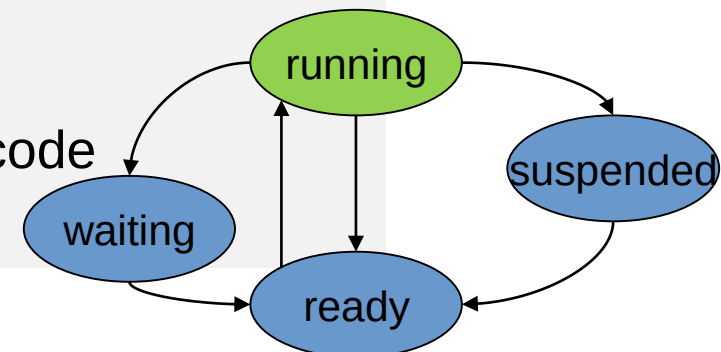
GetEvent(Receiver, &myMask)

```
if (myMask & ev1) {  
    ClearEvent(ev1);  
    //code for ev1  
}
```

//Some more code



Eventmask		
0	0	0



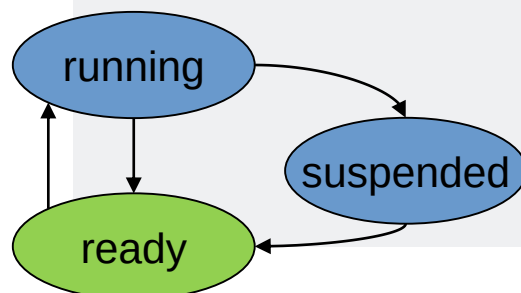
OSEK Event Sequence (2)

TASK(Sender)

//Execute some code

SetEvent(Receiver, ev1);

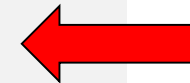
//Execute some code



TASK(Receiver)

//Execute some code

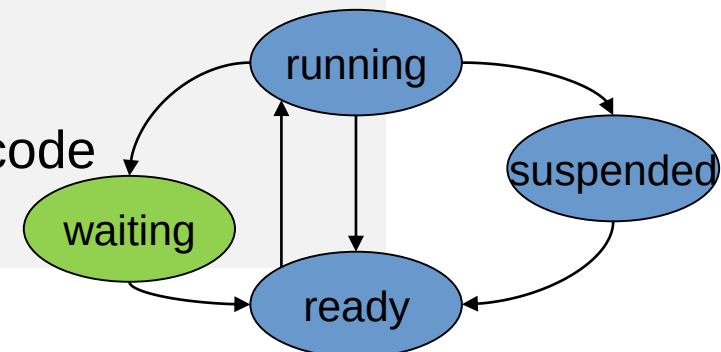
WaitEvent(ev1|ev2|ev3);



GetEvent(Receiver, &myMask)

```
if (myMask & ev1) {  
    ClearEvent(ev1);  
    //code for ev1  
}
```

//Some more code



Eventmask

0	0	0
---	---	---

OSEK Event Sequence (3)

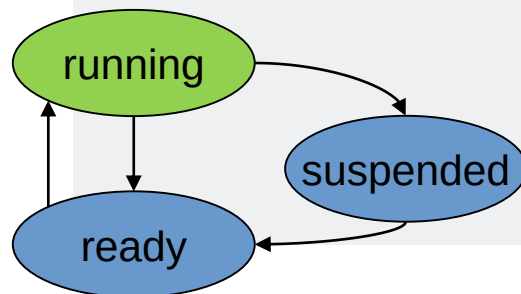
TASK(Sender)

//Execute some code



SetEvent(Receiver, ev1);

//Execute some code



TASK(Receiver)

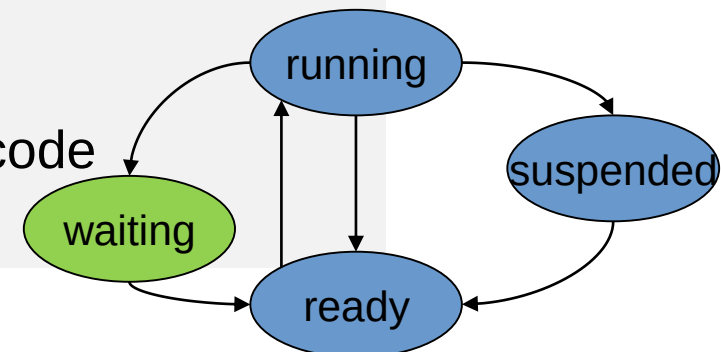
//Execute some code

WaitEvent(ev1|ev2|ev3);

GetEvent(Receiver, &myMask)

```
if (myMask & ev1) {  
    ClearEvent(ev1);  
    //code for ev1  
}
```

//Some more code



Eventmask

0	0	1
---	---	---

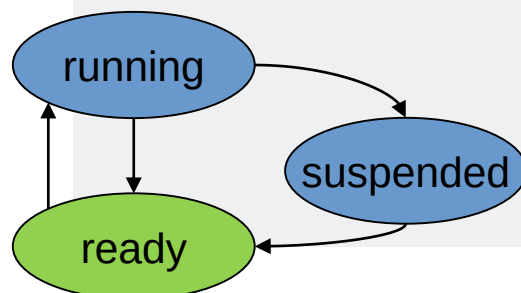
OSEK Event Sequence (4)

TASK(Sender)

//Execute some code

SetEvent(Receiver,ev1);

//Execute some code



TASK(Receiver)

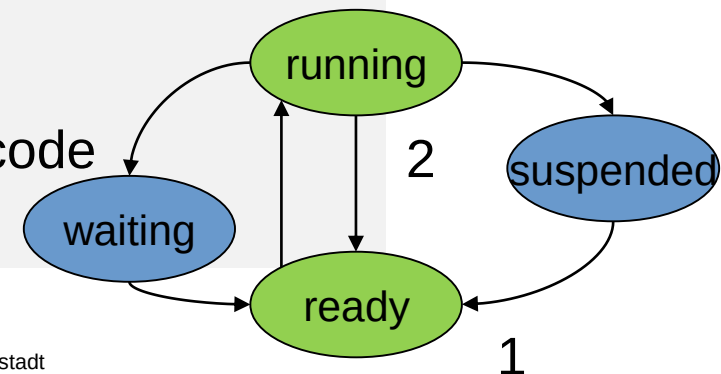
//Execute some code

WaitEvent(ev1|ev2|ev3);

GetEvent(Receiver,&myMask)

```
if (myMask &ev1) {  
    ClearEvent(ev1);  
    //code for ev1  
}
```

//Some more code



Eventmask

0	0	1
---	---	---

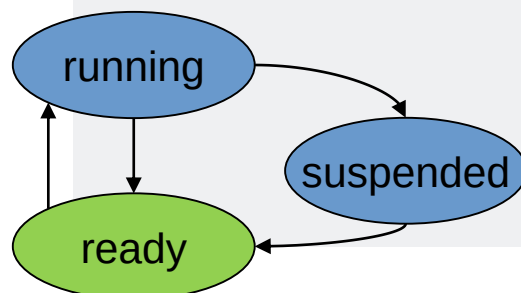
OSEK Event Sequence (5)

TASK(Sender)

//Execute some code

SetEvent(Receiver, ev1);

//Execute some code



TASK(Receiver)

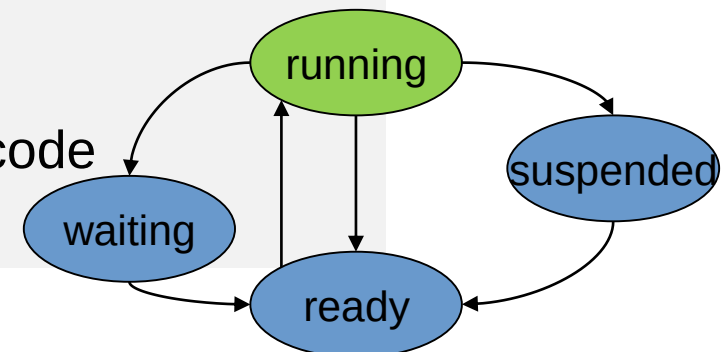
//Execute some code

WaitEvent(ev1|ev2|ev3);

GetEvent(Receiver, &myMask)

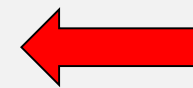
```
if (myMask & ev1) {  
    ClearEvent(ev1);  
    //code for ev1  
}
```

//Some more code



Eventmask

0	0	0
---	---	---



Some considerations about different implementation patterns – all checked

```
TASK(tsk_eventReceiver)
{
    EventMaskType ev = 0;
    while(1)
    {
        //Wait, read and clear the event
        WaitEvent(ev_button1 | ev_button2 | ev_button3 | ev_button4);
        GetEvent(tsk_eventReceiver, &ev);
        ClearEvent(ev);

        if (ev & ev_button1)
        {
            SEVEN_SetHex(++count);
        }

        if (ev & ev_button2)
        {
            count = 0;
            SEVEN_SetHex(count);
        }
    }
}
```

Reading and clearing the event are done in the shortest possible block. This reduces the risk of losing an event.

All events are checked in the loop, no need for extra buffering.

Order of if-statements may have side effect on behaviour, as developer only expected and tested one event at a time!

Some considerations about different implementation patterns – only one checked

```
TASK(tsk_eventReceiver)
{
    EventMaskType ev = 0;
    while(1)
    {
        //Wait, read and clear the event
        WaitEvent(ev_button1 | ev_button2 | ev_button3 | ev_button4);
        GetEvent(tsk_eventReceiver, &ev);
        ClearEvent(ev);

        if (ev & ev_button1)
        {
            SEVEN_SetHex(++count);
        } else if (ev & ev_button2)
        {
            count = 0;
            SEVEN_SetHex(count);
        }
    }
}
```

Reading and clearing the event are done in the shortest possible block. This reduces the risk of losing an event.

Only one event is checked. `ev_button2` might get lost, if both were fired at the same „tick“

Some considerations about different implementation patterns – only one checked

```
TASK(tsk_eventReceiver)
{
    EventMaskType ev = 0;
    while(1)
    {
        //Wait, read and clear the event
        WaitEvent(ev_button1 | ev_button2 | ev_button3 | ev_button4);
        GetEvent(tsk_eventReceiver, &ev);

        if (ev & ev_button1)
        {
            ClearEvent(ev_button1);
            SEVEN_SetHex(++count);
        } else if (ev & ev_button2)
        {
            ClearEvent(ev_button2);
            count = 0;
            SEVEN_SetHex(count);
        }
    }
}
```

Reading and clearing the event are done separately. This increases the risk of losing an event, as 1 | 1 remains 1.

Only one event is cleared, ev_button2 will be processed immediately afterwards in the second loop.

Some considerations about different implementation patterns – additional loop

```
TASK(tsk_eventReceiver)
{
    EventMaskType ev = 0;
    while(1)
    {
        //Wait, read and clear the event
        WaitEvent(ev_button1 | ev_button2 | ev_button3 | ev_button4);
        GetEvent(tsk_eventReceiver,&ev);
        ClearEvent(ev);

        EventMaskType localEvent = 0;
        while (0 != ev)
        {
            localEvent = getAndClearLowestBit(&ev)
            if (localEvent & ev_button1)
            {
                SEVEN_SetHex(++count);
            } else if (localEvent & ev_button2)
            {
                count = 0;
                SEVEN_SetHex(count);
            }
        }
    }
}
```

Reading and clearing the event are done in the shortest possible block. This reduces the risk of losing an event.

Only one bit of the event is processed per loop. Good for state machine functions.

Alarms

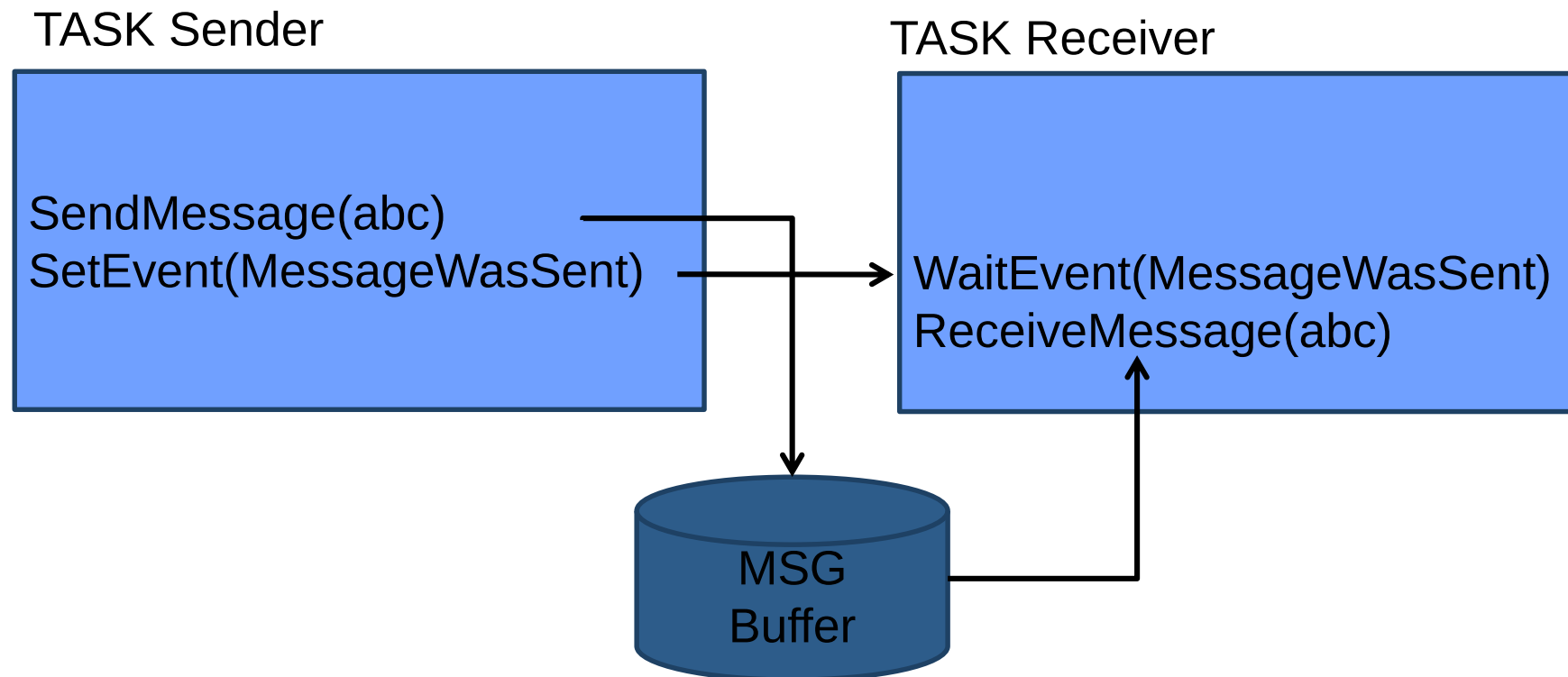
- Alarms are similar to events, but are time triggered.
- Alarms can be used to either start an event or a task or callback
-
- Typical applications include:
 - Perform a tasks every x ms (→ cyclic tasks)
 - Send an event x ms after a task has been finished (→ buffer time)
 - Send an event every x ms
 - Send an event at the absolute time x

Alarms, Events and Messages

- Until now, we used events and alarms to synchronize states between different tasks
 - Events and alarms can be considered as “software interrupts”
 - They do not transfer any data!
- ➔ additional mechanisms are required
- ➔ messages

Messages

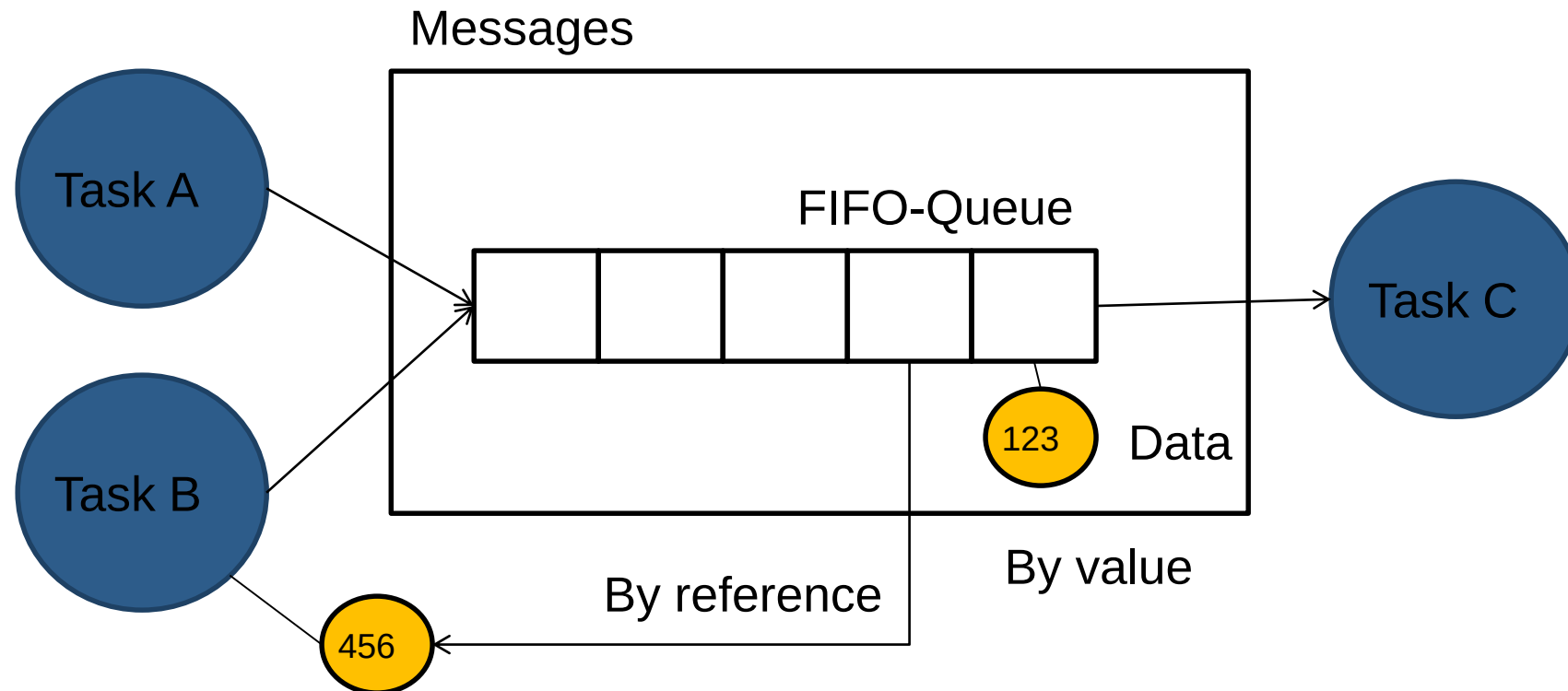
- In OSEK, “real” messages (unfortunately) do not exist.
- Instead, shared memory and events are combined.



Message Concept

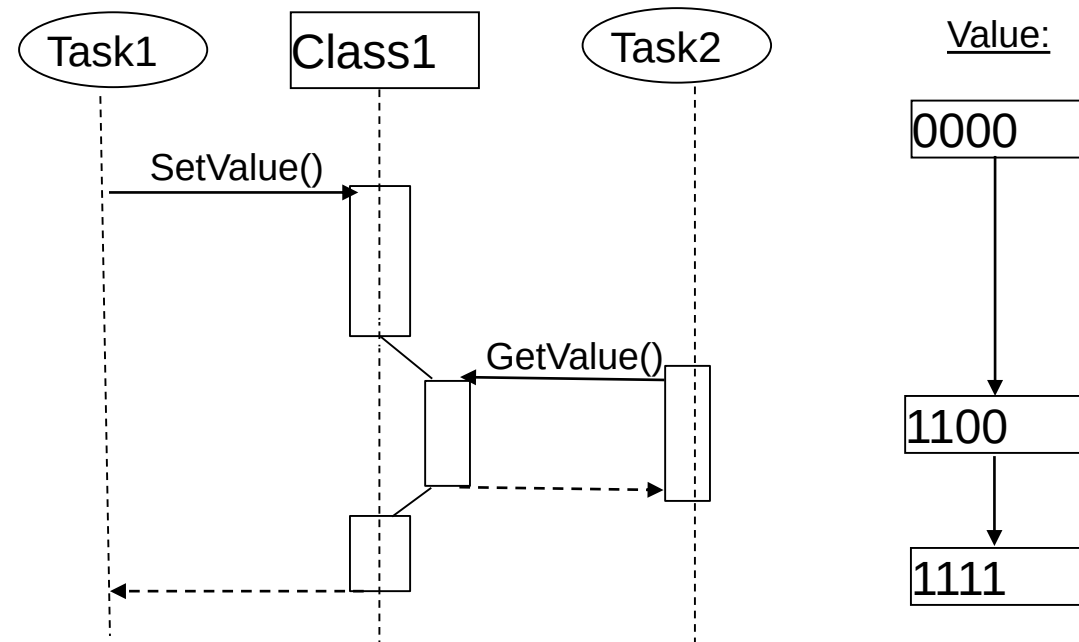
- Messages are used to exchange information between tasks containing data
- Messages are typically implemented using a FIFO queue
- Transmission usually supports n:m broadcasting
- Data can either be transmitted “by value” or “by reference”
- In OSEK, Messages can be combined with an event, i.e. an event is generated to wake up the task receiving the data.
- Messages can be internal (i.e. between tasks running on the same controller) or external (i.e. between two controllers connected via a network)

Message Concept



Problem: Shared Memory Areas

- Shared memory areas, i.e. several tasks addressing the same global variable are the most efficient way to transmit data between 2 tasks
- Unfortunately, it is also the most dangerous way to do it, unless the access is protected.



Shared memory pattern in Autosar (simplified)

Sender

```
ApplicationSenderFunc()  
{  
    struct data;  
  
    data = someCalculation();  
  
    //Write data to global pool  
    RTE_send_data(&data);  
}
```

```
RTE_send_data(struct* data)  
{  
    //Start critical section  
    GetRessource(res1);  
  
    POOL_globalObjectXYZ = *data; // This is very likely a non-atomic access  
                                   // and thus needs to be protected  
  
    //End critical section  
    ReleaseRessource(res1);  
  
    SendEvent(tsk_Receiver, ev_data);  
}
```

Shared memory pattern in Autosar (simplified)

Receiver

```
TASK(tsk_Receiver)
{
    while (1){
        WaitEvent(ev_data | ..... )
        GetEvent(tsk_Receiver, &myMask)

        if (myMask & ev_data){
            ClearEvent(ev_data);

            //Start critical section
            GetRessource(res1);

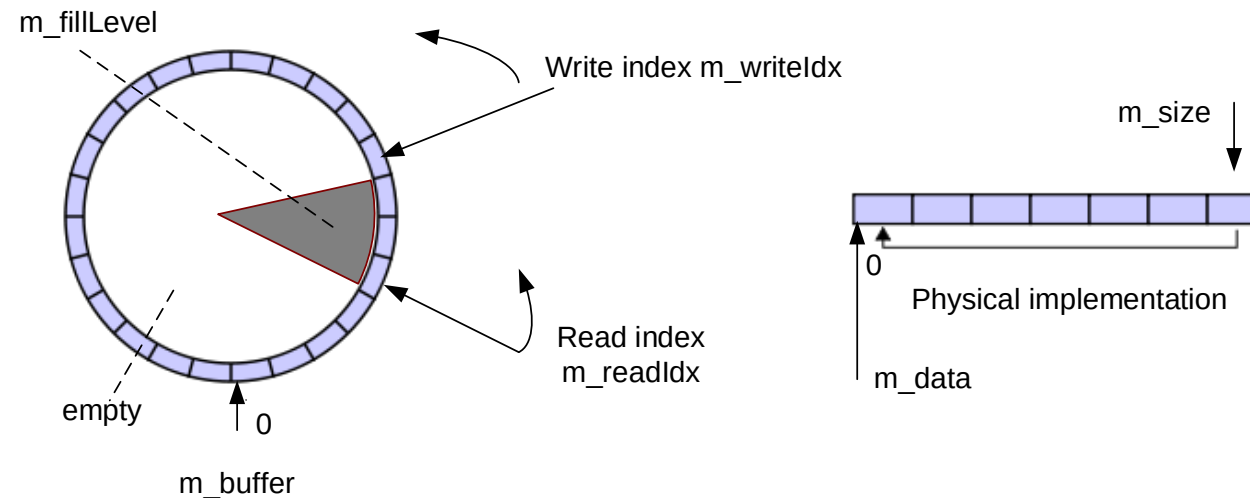
            struct data = POOL_globalObjectXYZ; // Again, non-atomic

            //End critical section
            ReleaseRessource(res1);

            ApplicationReceiverFunc(&data); // Note, the application get's the local copy!
                                           // Alternatively, the application must be inside
                                           // the critical region - long blocking!
        }
    }
}
```

Extension of the messaging concept

- Limitation of the simple global variable in the data pool: We need to read out the data before writing a new one, otherwise we will lose a data element
- Depending on the application this may be acceptable or even wanted or create a serious issue
 - Sensor distance data – probably only the latest is interesting
 - Communication protocol – we need to process every transmission!



Key pattern

Sender:

- Get Ressource
- Write to the ringbuffer
- Release Ressource
- Fire an event to receiver

Receiver:

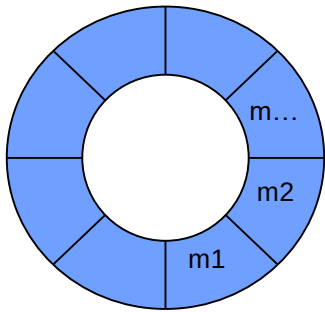
- Wait and check Event
- GetRessource
- Read from ringbuffer
- Release Ressource



Thins are pretty straight forward, if we have only one receiver and a constant message size! But how about multiple receiver communication and variable message size (e.g. transmission of strings)?

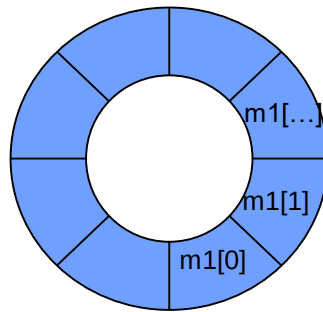
Buffer variants

Constant Message Size



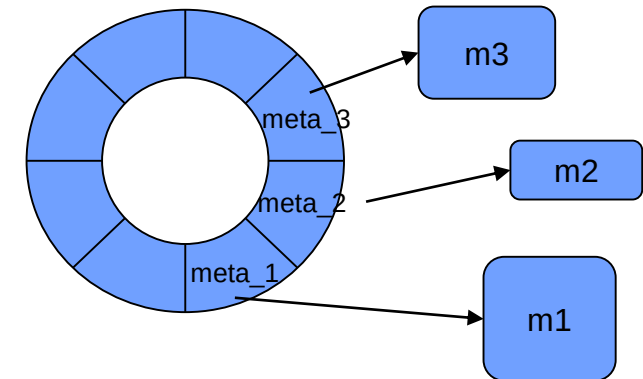
- Based on message_t buffer
- Constant message structure required
- Simple implementation

Variable Message Size Streaming Concept



- Based on uint8_t buffer
- Good for messages with EOP behavior (e.g. incoming UART protocol)
- EOP required to identify end of message stream

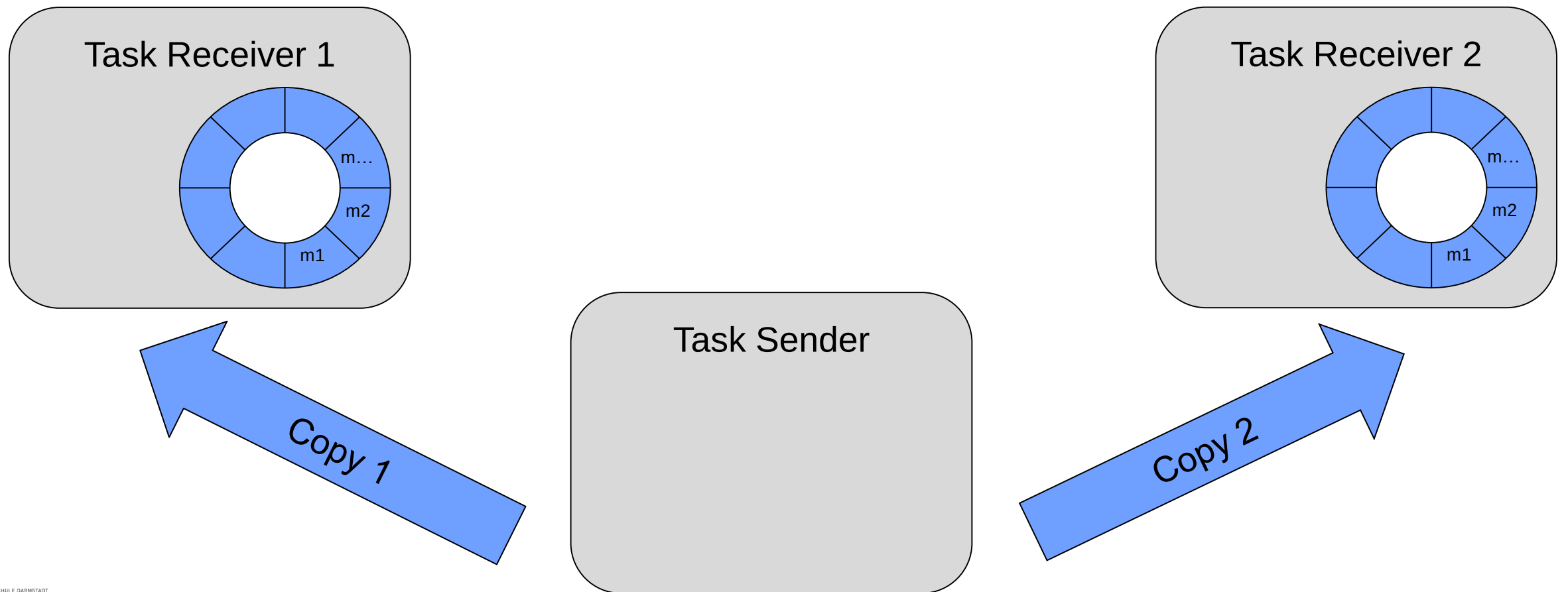
Variable Message Size Metadata Concept



- Based on metadata (size, pointer) structure
- Good for messages with flexible but known size (e.g. strings)
- Memory is allocated upon write and deallocated upon read

How about multiple receivers?

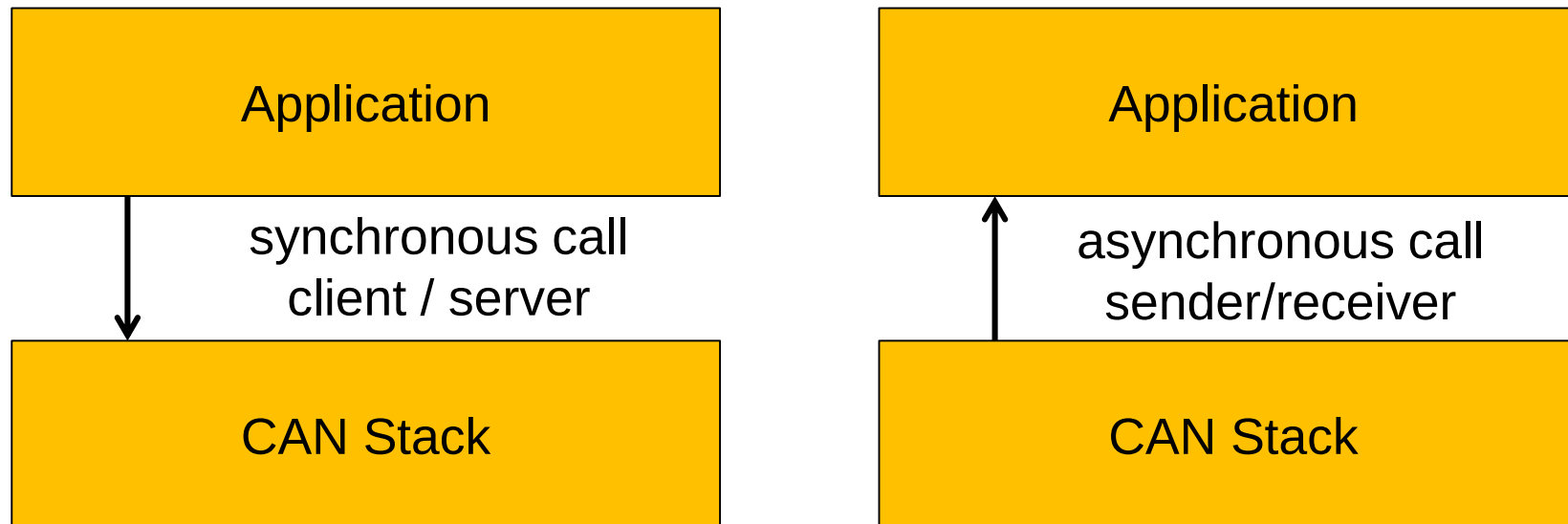
- The mailbox belongs to the receiver!
- Own copies increase runtime but reduce the risk for races / data inconsistencies
- Alternative: Transmission of references / pointers (envelope concept)



Example: Transport Protocol for CAN messages

Requirements:

- CAN messages are received via an ISR
- Every message is stored in a buffer
- The buffer checks, if a complete protocol is received
- Once a complete protocol is in, the content will be transferred to the application code



Example: Engine Control

Client/Server or Sender/Receiver?



What happens, if we replace the CAN with an ADC?

Patterns

Sender / Receiver

- Asynchronous sending and reception of data
- Involves typically several context areas
- Often used for communication interfaces
- We have a significant latency between sending and receiving of data
- Communication can be triggered bottom up
- More complex, events and buffers required

Client / Server

- Synchronous transfer, comparable to normal function call
- Involves one context area
- Function provides (parameter) and / or receives (return value, reference parameters) data
- Typically triggered by application (top down)
- No significant latency between function call and return

Design example

Which input signals
do we have to
process?

Information
exchange between
tasks?



Which tasks do we
need?

Static architecture?

Wrap-Up

- The step from decoupled to coupled systems
- Mechanisms to control the synchronisation between tasks
 - Events
 - Alarms
- Mechanisms to communicate between tasks
 - Message concepts
 - Shared Memory